

D11. Testing en aplicaciones backend

Sitio: [Agencia de Habilidades para el Futuro](#)
Curso: Desarrollo de Sistemas Web (Back End) - 2° E - 1C2026
Libro: D11. Testing en aplicaciones backend

Imprimido por: Adrian Madroñal
Día: viernes, 19 de junio de 2026, 10:16

Descripción

Tabla de contenidos

1. Preguntas Iniciales

2. Introducción

3. Principios del testing

- 3.1. Importancia de las pruebas y la automatización
- 3.2. Ciclo de vida del software
- 3.3. Calidad y defectos

4. Test Driven Development

- 4.1. Ciclo Red, Green, Refactor
- 4.2. Pasos de TDD
- 4.3. Ventajas
- 4.4. Historias de usuario y criterios de aceptación
- 4.5. Tipos de pruebas de software o tipos de testing
- 4.6. Herramientas de TDD
- 4.7. Testing unitario
- 4.8. Otros tests

5. JEST

- 5.1. Nuestro primer test
- 5.2. Matchers
- 5.3. Configuraciones y desmontaje
- 5.4. Grouping test
- 5.5. Mocking
- 5.6. Pruebas de funcionamiento asíncronas

6. Ejemplo práctico de unit testing con Jest

7. Ejemplo de TDD con Node

- 7.1. Setup inicial del proyecto
- 7.2. Cómo hacer la primera prueba con TDD
- 7.3. Algunas pruebas
- 7.4. Unit tests VS Integration tests
- 7.5. Unit Testing aplicando TDD en Node
- 7.6. Integration Testing aplicando TDD en Node
- 7.7. Estructura de Node JS con Express aplicando MVC

Preguntas iniciales

El testing es una de las actividades que desarrolla un programador, por lo que es importante conocer herramientas de testing para hacer nuestras propias pruebas.

¿Qué temas abordaremos?

Fundamentalmente los principios del testing, la importancia de las pruebas y la automatización en el ciclo de vida del software, sobre la calidad y los defectos, el Test Driven Development (TDD) y su ciclo Red, Green, Refactor.

Discutiremos los diferentes tipos de pruebas de software, las herramientas de TDD y los diferentes tipos de tests y su importancia, incluyendo el testing unitario y el testing de integración. Además, presentaremos una introducción al unit testing con Jest.

Preguntas Orientadoras

- ¿Por qué es importante realizar pruebas y cómo se pueden implementar en una aplicación Node.js?
- ¿Qué es el Test Driven Development (TDD) y cómo se relaciona con el ciclo de vida del software?
- ¿Qué ventajas tiene TDD?
- ¿Cómo se relacionan las historias de usuario y los criterios de aceptación con el testing?
- ¿Cuáles son los diferentes tipos de pruebas de software y cuál es su importancia?
- ¿Qué es el testing de integración y cómo se realiza?
- ¿Qué es el unit testing y cómo se puede implementar con Jest?

Introducción

El **Testing de Software** es toda una disciplina en la ingeniería de software permite tener procesos, métodos de trabajo y herramientas para identificar defectos en el software alcanzando un proceso de estabilidad del mismo.

El Testing no es una actividad que se piensa al final del desarrollo del software, sino que va paralelo a este.

El Testing de Software nace aproximadamente en el año 1960 a partir de la crisis del desarrollo del software, cuando empiezan a desarrollar los primeros softwares para el Departamento de Defensa de los Estados Unidos.

Objetivos del Testing

- Encontrar defectos o bugs con el fin de ser subsanados.
- Aumentar la confianza en el nivel de calidad del software.
- En base a la calidad, permite llevar a considerar la toma de decisiones.
- Evitar el aumento de defectos o Bugs en la APP testada.

Principios de testing

El ISTQB International Software Testing Qualifications Board propone los siete principios del testing.

1 La ejecución de pruebas demuestra la presencia de defectos

El testing nos permite demostrar la existencia de incidentes en un producto, no su ausencia. Tras el reporte de un incidente y su consecuente restauración, es posible reducir la probabilidad de que incidentes aún no descubiertos persistan en el sistema, pero la inexistencia absoluta de incidentes no sólo resulta imposible de comprobarse sino que además es una posibilidad sumamente improbable.

2 El testing exhaustivo no es posible

Exceptuando casos puntuales, probar cada combinación posible de datos y acciones en todas las funcionalidades de un software generalmente no es una opción viable, ya sea por cuestiones de tiempo, costos o recursos.

Por este motivo, al estructurar nuestras estrategias y diseñar los casos de pruebas, resulta muy conveniente considerar estos factores, para poder enfocarnos en ejecutar las verificaciones más significativas y que tengan mayor impacto.

3 Pruebas tempranas

Cuando el proceso de testing comienza en las etapas más tempranas del desarrollo de software, esta alianza resulta extremadamente beneficiosa, pues permite detectar incidentes en los albores del producto. El costo de reparación de estos incidentes sería significativamente más elevado si recién fueran detectados en fases posteriores.

4 Agrupación de defectos

Generalmente, hay ciertos módulos y funcionalidades que son más proclives a presentar incidencias (de mayor prioridad) en comparación al resto de las partes que conforman un producto. Este fundamento está relacionado con la Regla 80/20, que afirma que aproximadamente el 80% de las consecuencias emergen del 20% de las causas.

En términos más precisos, esto podría traducirse de la siguiente manera: no todos los componentes de un software poseen el mismo nivel de relevancia, por lo que el enfoque más propicio en el momento de elaborar nuestra estrategia de pruebas es concentrar nuestros esfuerzos justamente en esas partes más cruciales, para detectar aquellos incidentes que puedan resultar más urgentes de resolver.

5 Paradoja del pesticida

Ejecutar los mismos tests en una misma parte estable de un sistema repetidas veces es una práctica que tiende a dificultar la detección de nuevos incidentes que aún no han sido descubiertos.

Por consiguiente, para aumentar las posibilidades de detección de incidentes, es imprescindible revisar y actualizar de manera constante nuestra estrategia de pruebas y asegurarnos de explorar las diversas partes que componen el producto con mayor profundidad.

6 El testing depende del contexto

La estrategia y el tipo de pruebas serán seleccionados en función del sistema y a los entornos que se pretenden verificar. Por ejemplo, un software médico no será probado de la misma forma que un sistema de e-commerce.

Los procedimientos estructurados, así como los casos diseñados, serán seleccionados teniendo en consideración todos y cada uno de estos factores.

7 Falacia de ausencia de incidentes

Supongamos que todos los incidentes detectados en un sistema dado son corregidos. A continuación, se realiza un nuevo ciclo de pruebas, tras el cual no se detecta la presencia de nuevos incidentes.

La ausencia de nuevos errores detectados no implica necesariamente que el software sea útil, ya que la utilidad del mismo no es indicada por este parámetro, sino por la satisfacción de las expectativas del cliente que el producto pueda brindar.



Importancia

De las pruebas

Posibilita los procesos, métodos de trabajo y herramientas necesarias para garantizar la calidad de cualquier desarrollo.

De la automatización

Consiguen la eliminación de errores surgidos a raíz de las pruebas realizadas de forma manual.

Ciclo de vida del software

A lo largo del ciclo de vida del software se realizan distintas pruebas para garantizar que este cumpla con los requerimientos para los que fue diseñado y de la misma forma se encuentren procesos de mejora y optimización a medida que se desarrolla el software.

- Es necesario hacer pruebas en todas las fases del desarrollo de software ya que un error encontrado en una etapa tardía puede generar costos muy elevados.
- Errores detectados lo antes posible reducen los costos y son mucho más fáciles de corregir.
- El ciclo de vida permite que los errores se detecten lo antes posible y por lo tanto, permite a los desarrolladores concentrarse en la calidad del software, en los plazos de implementación y en los costos asociados.

Calidad y defectos

La calidad es una percepción entre lo deseado, analizado y lo que vamos a entregar. La calidad la define el cliente, si esa persona está satisfecha con lo entregado hasta ahí llega la calidad.

- **Verificación:** Es ir en cada etapa revisando que se cumpla lo propuesto por el cliente
- **Validación:** Antes de entregar al cliente, validamos que efectivamente el conjunto de requerimientos está siendo cumplido con lo entregado.
- **Anomalía:** La manifestación de un error en el software.
- **Error:** Una acción humana que produce un resultado incorrecto.
- **Defecto:** Imperfección o deficiencia, el cual no cumple sus requerimientos o especificaciones y necesita ser reparado o remplazado.
- **Fallo:** El cese de la habilidad de un producto de cumplir una función requerida o su inhabilidad de funcionar dentro de márgenes previamente especificados.
- **Problema:** Dificultad o incertidumbre experimentada por una o más personas, como resultado de un encuentro insatisfactorio con el sistema usado.

Test Driven Development

Test Driven Development o desarrollo dirigido por pruebas es una técnica para desarrollar software que consiste en ciclos cortos de código en los que primero se escribe una prueba automatizada que falle, después se hace lo mínimo necesario para que pase y finalmente refactor.

Fue creado por Kent Beck como parte de la metodología de eXtreme Programming (XP) con la promesa de tener varios beneficios como veremos más adelante.

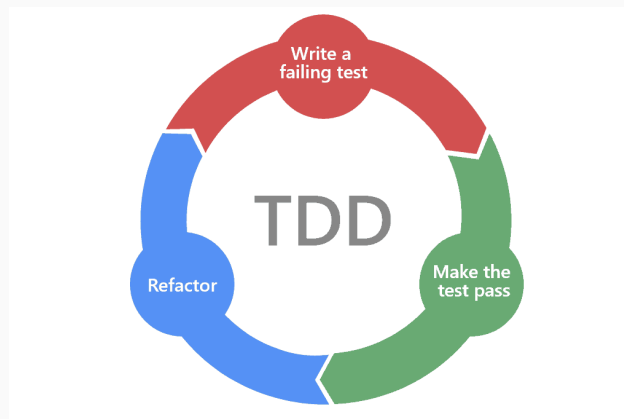
TDD es una técnica para diseñar software que se centra en tres pilares fundamentales:

- La implementación de las funciones justas que el cliente necesita y no más.
- La minimización del número de defectos que llegan al software en fase de producción.
- La producción de software modular, altamente reutilizable y preparado para el cambio.

Ciclo Red, Green, Refactor

El ciclo de Red, Green, Refactor es la esencia de TDD y consiste en:

- **RED:** Escribir la especificación del requisito (el ejemplo, el test). Comenzar creando una prueba automatizada que falle de un inicio. Por lo común, una prueba que falla tiene un color rojo en los tests runners.
- **GREEN:** Implementar el código según el test. Hacer lo mínimo necesario para que la prueba pase. Por lo común, una prueba que pasa tiene un color verde en un test runner.
- **REFACTOR (Refactorizar):** Refactorizar no significa reescribir el código; reescribir es más general que refactorizar. Refactorizar es modificar el diseño sin alterar su comportamiento. En este tercer paso del algoritmo TDD, rastreamos el código (también el del test) en busca de líneas duplicadas y las eliminamos refactorizando. Además, revisamos que el código cumpla con ciertos principios de diseño (por ejemplo S.O.L.I.D) y refactorizamos para que así sea.



Cuando hemos dado los tres pasos de la especificación que nos ocupa, tomamos la siguiente y volvemos a repetirlos.

Pasos de TDD

Estos son pasos generales con los que se aplica el TDD:

- Escribir el test.
- Ejecutar el test (esperando que falle).
- Escribir la implementación de ese test.
- Ejecutar el test de nuevo (esperando que pase).
- Refactorizar (limpiar código si es necesario).
- Repetir.

Ventajas

Las siguientes ventajas son:

- **Aumento de la confiabilidad:** al escribir pruebas antes del código, se asegura que el software funcione correctamente.
- **Mejora en la calidad del código:** al escribir pruebas específicas para el código, se pueden detectar problemas de diseño o errores de programación desde el principio, lo que ayuda a mejorar la calidad del código.
- **Mayor facilidad para factorizar:** al tener pruebas automatizadas, es más fácil realizar cambios en el código sin tener que preocuparse de romper algo.
- **Mayor rapidez en el desarrollo:** al escribir pruebas antes del código, se tiene una mejor comprensión de lo que se espera del software, lo que permite desarrollarlo más rápidamente. Escribir el ejemplo (test) antes que el código nos obliga a escribir el mínimo de funcionalidad necesaria, evitando sobrediseñar.
- **Mayor colaboración y comunicación:** al tener pruebas automatizadas, se pueden compartir entre los miembros del equipo, lo que facilita la colaboración y comunicación en el desarrollo. El trabajo en equipo se hace más fácil.
- **Mayor facilidad para detectar y corregir errores:** al tener pruebas automatizadas se detectan los errores de manera temprana y se pueden corregir de manera eficiente.
- **Código altamente reutilizable.**
- **Documentación:** cuando revisamos un proyecto desarrollado mediante TDD, nos damos cuenta de que los tests son la mejor documentación técnica que podemos consultar a la hora de entender qué misión cumple cada pieza del puzzle.

Historias de usuario y criterios de aceptación

Por lo común, el *stakeholder* o el *product owner* es quien nos proveerá de esta información porque es quién conoce al producto/negocio.

A continuación, las historias de usuario y criterios de aceptación que estaremos trabajando en:

Nuestro ejemplo

“Como vendedor, quiero registrar la información de mis productos en el sistema para consultarlos después.”

Criterios de aceptación:

- Guardar correctamente en una base de datos la información de un producto: Nombre, descripción, precio y stock.
- Validar que los datos del producto están completos antes de guardar. En caso de faltar un dato, retornar el mensaje "el valor [campo] es requerido".

Tener estos requisitos bien definidos antes de comenzar es crucial para trabajar con TDD de manera más efectiva.

Tipos de pruebas de software o tipos de testing

Existen diferentes tipos de pruebas de calidad de software que nos ayudan a mejorar lo que estamos desarrollando y pueden ser manuales o automáticas. Algunos de los tipos pruebas son:

- **Pruebas Funcionales:** Con este tipo de pruebas testeamos la funcionalidad de nuestro sistema o software, podemos hacernos preguntas sobre cómo funciona, qué debe estar haciendo, cómo están interactuando los usuarios. También conocidas como pruebas de caja negra. Estas pruebas nos ayudan a revisar todas las funciones del sistema y componentes con los que el usuario tiene una interacción directa.

Las pruebas funcionales se dividen en:

1. Pruebas unitarias
2. Pruebas de integración
3. Pruebas de sanidad
4. Pruebas de regresión
5. Pruebas de aceptación
6. Pruebas de sistema

- **Pruebas no-funcionales:** puede que tengamos un sistema funcionando, pero el usuario está experimentando otro tipo de problemas que no son detectados por las pruebas anteriores. Factores como lentitud, problemas en la combinación de colores provocando poca legibilidad o claridad, usabilidad, seguridad, son las que testeamos en con este tipo de pruebas.

Así que las pruebas no funcionales se enfocan en elementos de experiencia de usuario, diseño, seguridad y, en general, de calidad del sistema.

Algunas pruebas no funcionales existen son:

1. Pruebas de rendimiento
2. Pruebas de portabilidad
3. Pruebas de usabilidad
4. Pruebas de carga (load testing) nos permite identificar la cantidad de peticiones que un sistema puede soportar.
5. Pruebas de estrés: consiste en probar los límites que un sistema puede soportar. En este tipo de pruebas se suele enviar más peticiones de las que el software podría atender normalmente para saber el comportamiento de la aplicación.

- **Pruebas estructurales:** Mejor conocidas como pruebas de caja blanca porque conocemos todo nuestro stack de tecnología.

Acá estamos interesados en conocer cómo funciona nuestro sistema, no con la perspectiva del usuario sino como desarrollador, por eso con este tipo de pruebas es necesario conocer cómo está internamente implementado nuestro código, cómo funciona “por dentro”.

- **Pruebas de manejo de cambios:** En esta parte probamos nuevamente un componente o un módulo para verificar que no haya sido afectado por actualizaciones realizadas en otras partes de nuestro software y así evitamos que los usuarios puedan percatarse del error.

Herramientas de TDD

No hay una herramienta como tal para implementar la metodología de TDD. Más bien, existen herramientas que nos ayudan a escribir y correr nuestras pruebas de forma automatizada.

Algunas de las herramientas y frameworks

Las más populares para diferentes lenguajes:

- **JUnit para Java**
- **Jest para JavaScript:** Existen muchas alternativas pero Jest tiene una comunidad muy grande que ha creado más herramientas para mejorar aún más la experiencia de los desarrolladores. También es open source y tiene muy buen mantenimiento por sus creadores, de Facebook. Además, funciona tanto del lado del cliente (frontend) como de lado del servidor (backend).
- **En Python:** unittest, pytest o Hypothesis.
- **MiniTest para Ruby.**

Testing unitario

En testigo, una prueba unitaria es una forma de comprobar el correcto funcionamiento de una unidad de código.

Un test unitario está centrado en la mínima unidad de código (método, clase, etc.) que puede ser probada de manera aislada, no a nivel de sistema o de integración.

Para mantener un test y su infraestructura asociada centrada, evita probar cosas entre los diferentes niveles de aplicación o de sistema.

Los test unitarios también sirven como documentación del modelo. Esto es así porque iremos creando tests sobre cada comportamiento que desarrollaremos. De esta forma, si alguien quiere saber cómo funciona tu modelo, además de leer la documentación correspondiente, puede ver los tests funcionando, cambiarlos, hacer alguna prueba y ver qué saben hacer tus objetos.

Otro de los beneficios que presentan, es que los podemos ejecutar cuando queramos.

Todo test unitario debe ser:

- **Atómico:** significa que el test prueba la mínima cantidad de funcionalidad posible. Esto es, probará un solo comportamiento de un método de una clase.
- **Automatizable:** No debería requerirse una intervención manual
- **Completo:** Deben cubrir la mayor cantidad de código.
- **Repetible:** No se deben crear pruebas que sólo puedan ser ejecutadas una sola vez
- **Independiente:** La ejecución de una prueba no debe afectar a la ejecución de otra.
- **Inocuo** significa que no altera el estado del sistema. Al ejecutarlo una vez, produce exactamente el mismo resultado que al ejecutarlo veinte veces. No altera la base de datos, ni envía emails ni crea ficheros, ni los borra. Es como si no se hubiera ejecutado.
- **Rápido:** Un solo test tendría que ejecutarse en una pequeña fracción de segundo

Ventajas de Testing Unitario

- Fomentan el cambio
- Documenta el código
- Los errores están más acotados y son más fáciles de localizar

Limitaciones del Testing Unitario

- No se descubren todos los errores
- No descubrirán errores de integración, rendimiento o gráfico
- Solo Son efectivas si se combinan con otras pruebas

Cómo funciona el testing unitario

El proceso que se lleva a cabo consta de tres partes:

- **Arrange:** donde se definen los requisitos que debe cumplir el código principal.
- **Act:** el proceso de creación, donde vamos acumulando los resultados que analizaremos.
- **Assert:** se considera el momento en que comprobamos si los resultados agrupados son correctos o incorrectos. Dependiendo del resultado, se valida y continúa, o se repara, de forma que el error desaparezca.

Otros tests

Tests de Aceptación

Es un test que permite comprobar que el software cumple con un requisito de negocio. Es un ejemplo escrito con el lenguaje del cliente pero que puede ser ejecutado por la máquina.

Son realizadas principalmente por los usuarios con el apoyo del equipo del proyecto. El propósito es confirmar que el sistema está terminado, que desarrolla puntualmente las necesidades de la organización y que es aceptado por los usuarios finales.

Tests Funcionales

Todos los tests son en realidad funcionales. No obstante, cuando se habla del aspecto funcional, se distingue entre test funcional y test no funcional. Un test funcional es un subconjunto de los tests de aceptación. Es decir, comprueban alguna funcionalidad con valor de negocio. Los tests de aceptación tienen un ámbito mayor porque hay requerimientos de negocio que hablan de tiempos de respuesta, capacidad de carga de la aplicación, etc; cuestiones que van más allá de la funcionalidad. Un test funcional es un test de aceptación pero, uno de aceptación, no tiene por qué ser funcional.

Tests de Sistemas

Esta prueba tiene como objetivo verificar que se han integrado adecuadamente todos los elementos del sistema y que realizan las operaciones apropiadas funcionando como un todo. Es similar a la prueba de integración pero con un alcance mucho más amplio.

Es el mayor de los tests de integración, ya que integra varias partes del sistema. Se trata de un test que puede ir, incluso, de extremo a extremo de la aplicación o del sistema.

Los tests de sistema son muy frágiles en el sentido de que cualquier cambio en cualquiera de las partes que componen el sistema, puede romperlos. No es recomendable escribir un gran número de ellos por su fragilidad.

Testing de Integración

Su objetivo es identificar errores introducidos por la combinación de programas o componentes probados unitariamente, para asegurar que la comunicación, enlaces y los datos compartidos ocurran apropiadamente. Se diseñan para descubrir errores o completitud en las especificaciones de las interfaces.

En este nivel se asegura que las interfaces y ligas entre las partes del sistema trabajen apropiadamente. Antes de las pruebas de integración, los componentes tuvieron que haber pasado sus pruebas individuales, por lo que el enfoque ahora es sobre el flujo de control entre los módulos, y sobre los datos que son intercambiados entre ellos de manera independiente.

JEST

JEST es el *framework* de test de Facebook, con el que se hacen los test de React, pero también se puede usar con otros *frameworks* como Vue o Angular, incluso con VanillaJs. Es muy rápido y muy fácil de aprender si empiezas con los test.

¿Qué es un *framework* de pruebas unitarias?

Un *framework* de pruebas unitarias es una herramienta que nos permite escribir pruebas sobre un bloque de código, ejecutándolo bajo un entorno de javascript sin necesidad de interferir en el IDE ni en la propia aplicación.

Instalación

JavaScript

Copiar código 

```
npm install --save-dev jest
```

Introducción a JEST

Creamos un nuevo proyecto de node a este le llamaremos unit_test e instalamos Jest.

Nuestro primer test

Ahora que ya hemos configurado y instalado Jest, es el momento de iniciar a escribir nuestro primer test, pero primero tenemos que escribir un código básico que podamos probar en primer lugar.

Para eso tenemos que crear dos archivos uno llamado suma.js y el otro suma.spec.js.

El primer archivo es donde vamos a tener una función llamada suma y el segundo archivo es que contendrá nuestra primera prueba.

Esta será la función que usaremos para escribir nuestra primera prueba:

```
1  const suma = (numero1, numero2) => {  
2    return numero1 + numero2;  
3  };  
4  
5  module.exports = { suma };  
6
```

Estos son algunos casos de prueba muy básicos para la función suma

```
1  const { suma } = require("./suma");  
2  
3  test("Suma 2 + 2", () => {  
4    expect(suma(2, 2)).toBe(4);  
5  });  
6  
7  test("Suma 4 + 2", () => {  
8    expect(suma(4, 2)).toBe(6);  
9  });  
10
```

Ahora ejecutamos el comando en la terminal `npm run test` y nos debería de dar el siguiente resultado:

```
PS C:\Users\Owner\Desktop\Proyectos\Node\unit_test> npm run test  
> unit_test@1.0.0 test C:\Users\Owner\Desktop\Proyectos\Node\unit_test  
> jest  
  
PASS ./suma.spec.js  
  ✓ Suma 2 + 2 (5 ms)  
  ✓ Suma 4 + 2 (1 ms)  
  
Test Suites: 1 passed, 1 total  
Tests:      2 passed, 2 total  
Snapshots:  0 total  
Time:       3.786 s  
Ran all test suites.
```

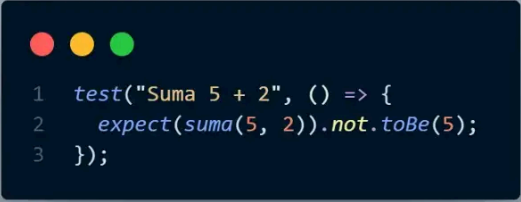
Matchers

Los **Matchers** nos permiten validar los resultados y valores de nuestras pruebas de diferentes maneras. Se utilizan más comúnmente para comprar el resultado de la función `expect()` con el valor pasado como argumento al `matchers`.

A continuación, se muestra una lista de los `Matchers` más comunes:

- **toBe** - compara igual.
- **toEqual** - compara el valor de dos variables o objetos.
- **toBeNull** - compara que el valor sea `null`.
- **toBeUndefined** - compara que el valor sea `undefined`.
- **toBeTruthy** - compara que el valor sea `true` (similar a un `if`).
- **toBeFalsy** - compara que el valor sea `false` (similar a un `if`).
- **toContain** - compara que el resultado del `expect()` contenga un valor.

Estos `matchers` también se pueden negar usando la declaración `not` ejemplo:



```
1 test("Suma 5 + 2", () => {  
2   expect(suma(5, 2)).not.toBe(5);  
3 });
```

Configuraciones y desmontaje

Casi siempre, al escribir pruebas, tendremos que realizar algún tipo de configuración como inicializar las variables, o la base de datos etc. Antes de que se ejecuten las pruebas y otra acción después de que hayan terminado. Jest nos ofrece dos formas diferentes de hacerlo.

One-Time Setup:

Esta opción nos sirve cuando sólo necesitamos realizar la configuración una vez. Para esto podemos usar las funciones `beforeAll()` y `afterAll()` que se ejecutarán antes de que comiencen las pruebas y después de que todas hayan terminado Ejemplo:

```
1  const { suma } = require("./suma");
2  let numero1;
3  beforeAll(() => {
4    numero1 = 2;
5  });
6
7  afterAll(() => {
8    numero1 = 0;
9  });
10
11 test("Suma 2 + 2", () => {
12   expect(suma(numero1, 2)).toBe(4);
13 });
```

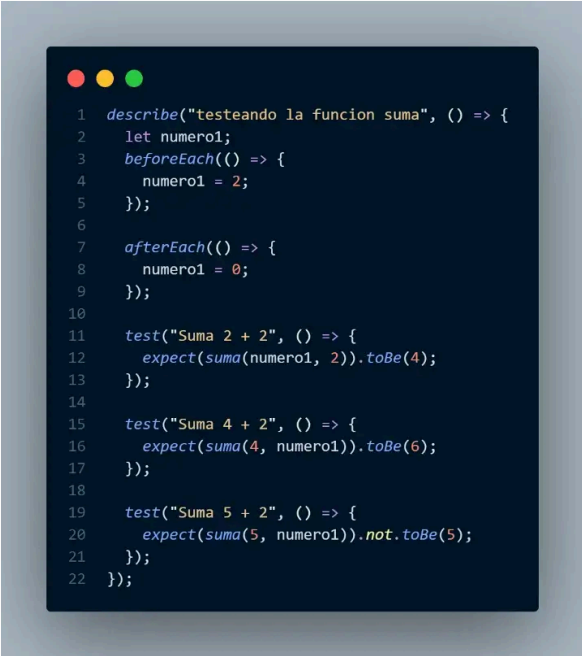
Repeating setup para cada prueba:

Si tenemos un proceso de configuración que se debe de ejecutar antes de cada prueba, entonces se debe de usar las funciones de `beforeEach()` y `afterEach()` ejemplo:

```
1  const { suma } = require("./suma");
2  let numero1;
3
4  beforeEach(() => {
5    numero1 = 2;
6  });
7
8  afterEach(() => {
9    numero1 = 0;
10 });
11
12 test("Suma 2 + 2", () => {
13   expect(suma(numero1, 2)).toBe(4);
14 });
15
16 test("Suma 4 + 2", () => {
17   expect(suma(4, numero1)).toBe(6);
18 });
19
20 test("Suma 5 + 2", () => {
21   expect(suma(5, numero1)).not.toBe(5);
22 });
```


Grouping test

También podemos agrupar las pruebas relacionadas para poder aislar las funciones de configuración y desmontaje. De igual forma nos puede ayudar a obtener una mejor descripción general de nuestros diferentes casos de prueba ejemplo:



```
1 describe("testeando la funcion suma", () => {
2   let numero1;
3   beforeEach(() => {
4     numero1 = 2;
5   });
6
7   afterEach(() => {
8     numero1 = 0;
9   });
10
11   test("Suma 2 + 2", () => {
12     expect(suma(numero1, 2)).toBe(4);
13   });
14
15   test("Suma 4 + 2", () => {
16     expect(suma(4, numero1)).toBe(6);
17   });
18
19   test("Suma 5 + 2", () => {
20     expect(suma(5, numero1)).not.toBe(5);
21   });
22 });
```

Mocking

Mocking es crear objetos, servicios o funciones que simulen comportamiento de objetos reales y juegan un papel vital en las pruebas. El objetivo es reemplazar algo que no controlamos cómo ser un api o un servicio externo con algo que tenga todas las características que necesitamos. También nos ayuda a inspeccionar información sobre nuestro código, como ejemplo si ya se ha llamado a una función y qué parámetros se utilizaron.

[[AGREGAR IMAGEN 2 PAG 21]]

Pruebas de funcionamiento asíncronas

Es común escribir funciones asíncronas en JavaScript utilizando promesas o *callbacks*. El problema de probar código asíncrono es saber cuando el código que se está probando está realmente completo, por suerte Jest tiene varias formas de manejar esto.

Promises

Probar las promesas es sencillo en Jest. Simplemente devuelva la promesa y Jest esperará a que se resuelva. Si la promesa falla, la prueba también fallará automáticamente.

Tenemos nuestra función asíncrona que devuelve una promesa, esta función lo que hace básicamente es recibir un *string* y ordenarlo al revés:

```
1  const stringAlReves = (str) => {
2    return new Promise((resolve, reject) => {
3      if (!str) reject("string Vacio");
4      resolve(str.split("").reverse().join(""));
5    });
6  };
7
8  module.exports = stringAlReves;
```

Nuestra prueba quedaría de la siguiente manera:

```
1  const stringAlReves = require("./string");
2
3  test('String la Reves 'Hola' igual a 'aloH'', () => {
4    return stringAlReves("Hola").then((result) => {
5      expect(result).toBe("aloH");
6    });
7  });
```

También podemos detectar promesas rechazadas utilizando la función `catch()`, ejemplo:

```
1  test("Probando la funcion enviando un string vacio y entrando al catch", ()
2    => {
3    return stringAlReves().catch((error) => {
4      expect(error).toMatch("string Vacio");
5    });
6  });
```

Async Await:

También podemos usar `async await` para probar promesas. Ejemplo:



```
1 test("Probando la funcion usando async await ", async () => {  
2   const result = await stringALReves("Hola");  
3   expect(result).toBe("aloH");  
4 });
```

Ejemplo práctico de unit testing con Jest

Set Up

Antes de empezar, vamos a preparar todo lo que necesitamos.

Adentro de una nueva carpeta, vamos a inicializar npm.

JavaScript

Copiar código 

```
npm init -y
```

Posteriormente, vamos a instalar jest. Como no necesitamos que sea parte del bundle de producción, vamos a instalarlo como dependencia de desarrollo.

JavaScript

Copiar código 

```
npm install --save-dev jest
```

Y para poder correr los tests, vamos a crear el comando test en el package.json (o modificarlo si tenemos el que viene por default con npm).

Así deberían quedar los scripts en package.json:

JavaScript

Copiar código 

```
"scripts": {  
  "test": "jest"  
},
```

Finalmente, vamos a crear el archivo donde vamos a escribir el código y el archivo donde vamos a escribir los tests:

- `fizzbuzz.js`
- `fizzbuzz.test.js`

Aplicación

Con el propósito de centrarnos en la parte técnica de los tests, vamos a crear un programa muy conocido que se llama fizzBuzz. Básicamente consiste en una función que recibe un número y devuelve:

- Fizz, si el número es divisible por 3.
- Buzz, si el número es divisible por 5.
- FizzBuzz, si el número es divisible por 3 y por 5.

- El mismo número, si el número no es divisible ni por 3 ni por 5.

Como vemos, la funcionalidad a implementar es muy sencilla. Esto está bueno para no tener que pensar en una lógica complicada y poder centrarnos en lo que hoy nos importa: los tests.

El programa consiste en 10 líneas muy sencillas de leer y entender:

JavaScript

Copiar código 

```
// fizzbuzz.js
function fizzBuzz(n) {
  const isDivisibleBy3 = n % 3 === 0;
  const isDivisibleBy5 = n % 5 === 0;

  if (isDivisibleBy3 && isDivisibleBy5) return 'FizzBuzz';
  if (isDivisibleBy3) return 'Fizz';
  if (isDivisibleBy5) return 'Buzz';

  return n;
}

module.exports = fizzBuzz;
```

Tests

La función describe, nos sirve para crear una suite de tests. Recibe un string como primer parámetro que nos sirve para describir qué es lo que testea la suite, y una función como segundo parámetro donde vamos a ir metiendo los tests más adelante.

Una suite de tests, es simplemente un conjunto de tests que testean distintas partes o distintos casos de la misma funcionalidad.

JavaScript

Copiar código 

```
describe('descripción', () => {});
```

Los tests, se crean con la función it, que es muy similar a describe. Recibe un string como primer parámetro, que nos sirve para describir qué cosa específica estamos testeando, y una función como segundo parámetro, que es el test en sí.

Además, vamos a hacer uso de la función expect, que recibe solamente un parámetro. Ese parámetro puede ser de cualquier tipo y es el valor que estamos testeando.

Luego, a la función expect le podemos encadenar otras funciones para verificar que el valor sea el esperado. Esos métodos que le encadenamos a expect, se llaman matchers. Con ellos podemos comprobar cosas como que el valor sea truthy o falsy, que sea mayor o menor a algo, que arroje una excepción, y un largo, largo etc. Para ver la lista completa podemos entrar a la documentación de Jest: <https://jestjs.io/docs/expect>.

Veamos un ejemplo simple que nos va a ayudar a entender todo esto que estuvimos hablando sobre la estructura de los tests:

JavaScript

Copiar código 

```
describe('Testeando true', () => {  
  it('should be truthy', () => {  
    expect(true).toBeTruthy();  
  });  
  
  it('should not be falsy', () => {  
    expect(true).not.toBeFalsy();  
  });  
});
```

Testeando Fizzbuzz

En el archivo `fizzbuzz.test.js`, vamos a importar nuestra función `fizzBuzz` y a crear nuestra primera suite de tests. Vamos a testear que la función `fizzBuzz` nos devuelva el mismo número que recibió, si dicho número no es divisible ni por 3 ni por 5.

JavaScript

Copiar código 

```
const fizzBuzz = require('./fizzbuzz');  
describe('FizzBuzz', () => {  
  it('should return number when number is not divisible by either 3 nor 5', () => {  
    expect(fizzBuzz(1)).toBe(1);  
    expect(fizzBuzz(2)).toBe(2);  
  });  
});
```

Como vemos, a la función `expect` le pasamos el resultado que devuelve `fizzBuzz` y le encadenamos el método `toBe`.

Con esto estamos diciendo algo así: “Esperamos (`expect`) que la función `fizzBuzz` con el parámetro 1, devuelva (`toBe`) 1”.

Ejecutemos los tests...

JavaScript

Copiar código 

```
npm test
```

Deberían salir todos en verde:

```
> jest@1.0.0 test
> jest

PASS ./fizzbuzz.test.js
  FizzBuzz
    ✓ should return number when number is not divisible by either 3 nor 5 (2 ms)
)

Test Suites: 1 passed, 1 total
Tests: 1 passed, 1 total
Snapshots: 0 total
Time: 0.329 s, estimated 1 s
Ran all test suites.
```

Para terminar, vamos a testear los siguientes 3 casos.

JavaScript

Copiar código 

```
const fizzBuzz = require('./fizzbuzz');

describe('FizzBuzz', () => {
  it('should return number when number is not divisible by neither 3 nor 5', () => {
    expect(fizzBuzz(1)).toBe(1);
    expect(fizzBuzz(2)).toBe(2);
  });

  it('should return Fizz when number is divisible by 3', () => {
    expect(fizzBuzz(3)).toBe('Fizz');
    expect(fizzBuzz(6)).toBe('Fizz');
  });

  it('should return Buzz when number is divisible by 5', () => {
    expect(fizzBuzz(5)).toBe('Buzz');
    expect(fizzBuzz(10)).toBe('Buzz');
  });

  it('should return FizzBuzz when number is divisible by both 3 and 5', () => {
    expect(fizzBuzz(15)).toBe('FizzBuzz');
    expect(fizzBuzz(30)).toBe('FizzBuzz');
  });
});
```

Ahora pasemos los tests a lenguaje humano...

1. `expect(fizzBuzz(3)).toBe('Fizz')`: Esperamos (expect) que la función `fizzBuzz` con el parámetro 3, devuelva (toBe) Fizz
2. `expect(fizzBuzz(5)).toBe('Buzz')`: Esperamos (expect) que la función `fizzBuzz` con el parámetro 5, devuelva (toBe) Buzz
3. `expect(fizzBuzz(15)).toBe('FizzBuzz')`: Esperamos (expect) que la función `fizzBuzz` con el parámetro 15, devuelva (toBe) FizzBuzz

Volvamos a correr los tests para verificar que todo está pasando:

JavaScript

Copiar código 


```
npm test
```

Sintaxis:

```
> jest@1.0.0 test
> jest

PASS ./fizzbuzz.test.js
  FizzBuzz
    ✓ should return number when number is not divisible by neither 3 nor 5 (2 ms)
    ✓ should return Fizz when number is divisible by 3
    ✓ should return Buzz when number is divisible by 5
    ✓ should return FizzBuzz when number is divisible by both 3 and 5

Test Suites: 1 passed, 1 total
Tests:       4 passed, 4 total
Snapshots:   0 total
Time:        0.34 s, estimated 1 s
Ran all test suites.
```

Ejemplo de TDD con Node

A continuación veremos un ejemplo creando una API REST usando Node JS y Express y aplicando TDD (Test Driven Development) paso a paso y desde cero.

Esto incluye habilidades de cómo crear Unit Tests (pruebas unitarias) e Integration Tests (pruebas de integración) usando Jest como principal herramienta de testing.

Setup inicial del proyecto

Comenzamos creando un nuevo proyecto de Node yendo a la terminal y nos posicionamos donde queramos tener el proyecto.

Creamos una [carpeta products-tdd](#) y ejecutamos:

JavaScript

Copiar código 

```
npm init -y
```

El **flag -y** le indica a **npm** que asigne los valores por default al **package.json** generado.

Ahora a instalar las dependencias:

JavaScript

Copiar código 

```
npm i express mongoose dotenv -S
```

Después instalamos las dependencias de desarrollo, aquellas que sólo usaremos en nuestra computadora local, pero que no requerimos en un ambiente de producción:

JavaScript

Copiar código 

```
npm i mongodb-memory-server jest supertest -D
```

Entonces:

- Mongodb-memory-server lo vamos a usar para las pruebas de integración como veremos más adelante.
- Jest como test runner, para crear mocks y assertions.
- Supertest como herramienta para probar los endpoints de la api.

Por último, lo mínimo necesario que requerimos para comenzar nuestra API es levantar el server. Creamos un nuevo archivo `server.js` en raíz del proyecto. Dentro inicializamos una instancia de Express y levantamos el server.

JavaScript

Copiar código 

```
const express = require('express');

const app = express();

const port = 8080;

app.listen(() => console.log(`listening on port ${port}`));
```

Ahora ejecutamos en la terminal

JavaScript

Copiar código 

```
node server.js
```

Estando en la raíz del proyecto y ya tenemos el server en el puerto 8080. Con esto ya podemos comenzar a desarrollar la API.

Cómo hacer la primera prueba

Recordemos que TDD consta de 3 fases:

- **RED:** Comenzar con una prueba que falle.
- **GREEN:** Hacer que la prueba pase con el mínimo esfuerzo.
- **REFACTOR:** Aplicar buenas prácticas de desarrollo (SOLID, Clean Code, DRY, etc).

Veamos cómo iniciar nuestra primera prueba en TDD con Node y Express.

En base a este criterio de aceptación:

“Guardar correctamente en una base de datos la información de un producto: Nombre, descripción y precio.”

Vamos a hacer la prueba más sencilla posible en la que ejecutaremos un endpoint y esperaremos una respuesta de 201 (stored) y mensaje con el nuevo producto registrado.

Ya que Node y Express son flexibles en cuanto a la estructura de archivos, nosotros somos libres de estructurar como deseemos. De momento, vamos a ir creando el código más simple posible y conforme avancemos en las pruebas y refactors, evolucionaremos a un patrón MVC.

En cuanto a las pruebas, vamos a crear una carpeta `__tests__` que jest reconoce y dentro colocamos un `products.test.js`.

Dentro vamos a importar la dependencia supertest que usaremos para probar nuestra API.

Al ejecutar supertest necesitamos pasarle por parámetro la instancia de express que contiene los endpoints definidos, por lo que también vamos a importar la constante app que definimos en el archivo `server.js`.

JavaScript

Copiar código

```
const request = require('supertest');

const { app } = require('../server');

// aqui colocaremos las pruebas...
```

Para que pueda funcionar el require de app, necesitamos exportarlo de server.js, por lo que lo modificamos `server.js` como sigue:

JavaScript

Copiar código

```
const express = require('express');

const app = express();

const port = 8080;

if (require.main === module) {
  app.listen(() => console.log('listening on port ${port}'));
}

module.exports.app = app;
```

Además, le he agregamos lógica para que inicialice el server sólo si este archivo lo estamos ejecutando con Node del modo node server.js, de lo contrario, no entra al if y por lo tanto, no inicializa el server. Esto es importante porque en las pruebas quien va a inicializar el server es supertest. Al final exportamos la propiedad app igual a la constante app que es la instancia de express.

Ahora regresamos a nuestra prueba y definimos la estructura general como:

JavaScript

Copiar código 

```
describe('POST /products', () => {
  test('should store a new product', async () => {
    // aqui haremos la primera prueba
  });
});
```

Ahora dentro del bloque test, ejecutaremos supertest. Ya que supertest soporta ejecutarlo en modo Promise (promesa), usamos async await para resolverla.

JavaScript

Copiar código 

```
describe('POST /products', () => {
  test('should store a new product', async () => {
    await request(app)
      .post('/products')
      .set('Accept', 'application/json')
      .expect('Content-Type', /json/)
      .expect(201);
  });
});
```

Esta es nuestra primera versión de prueba para el endpoint /users que aún no hemos definido en la api.

Para ejecutar la prueba, vamos al archivo `package.json` de nuevo y modificamos el script de test como "test": "jest", quedando como:

JavaScript

Copiar código

```
"scripts": {
  "test": "jest"
},
```

En la terminal ejecutamos

JavaScript

Copiar código

```
npm run test --watch
```

Para que se quede en modo observación y comprobamos que la prueba falle (fase red).

Obtenemos como resultado:

```
> products-tdd@1.0.0 test
> jest

FAIL __tests__/products.test.js
  POST /products
    ✕ should store a new product (24 ms)

● POST /products › should store a new product

  expected "Content-Type" matching /json/, got "text/html; charset=utf-8"

   8 |         .post('/products')
   9 |         .set('Accept', 'application/json')
>  10 |         .expect('Content-Type', /json/)
      |         ^
   11 |         .expect(201);
   12 |       });
   13 |     });

at Object.expect ( __tests__/products.test.js:10:8)
-----
at Test._assertHeader (node_modules/supertest/lib/test.js:232:16)
at node_modules/supertest/lib/test.js:308:13
at Test._assertFunction (node_modules/supertest/lib/test.js:285:13)
at Test.assert (node_modules/supertest/lib/test.js:164:23)
at Server.localAssert (node_modules/supertest/lib/test.js:120:14)

Test Suites: 1 failed, 1 total
Tests:       1 failed, 1 total
Snapshots:   0 total
Time:        0.522 s
Ran all test suites.
```

Nos dice que falla porque esperaba una respuesta de tipo JSON y estamos retornando una con HTML que es la respuesta default de express al ejecutar un endpoint no definido.

Ahora pasamos a hacer lo mínimo para resolver este error.

En `server.js`, agregamos la definición del endpoint `/products` y retornamos una respuesta tipo JSON que es lo mínimo necesario para que la prueba continúe.

JavaScript

Copiar código

```
app.post('/products', (req, res) => {
  res.json();
});
```

Quedando como:

JavaScript

Copiar código 

```
const express = require('express');

const app = express();

const port = 8080;

app.post('/products', (req, res) => {
  res.json();
});

if (require.main === module) {
  app.listen(() => console.log(`listening on port ${port}`));
}

module.exports.app = app;
```

Recordemos que necesitamos hacer lo MÍNIMO necesario para que las pruebas pasen, más adelante haremos los refactors necesarios como separación de código.

Ahora la prueba falla por el estatus code:



Entonces modificamos la respuesta del endpoint:

JavaScript

Copiar código 

```
app.post('/products', (req, res) => {
  res.status(201).json();
});
```

Y ahora si, la prueba ya pasa:


```
> products-tdd@1.0.0 test
> jest

PASS  __tests__/products.test.js
  POST /products
    ✓ should store a new product (29 ms)

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   0 total
Time:        0.518 s, estimated 1 s
Ran all test suites.
```

Pero apenas iniciamos. Ahora vamos a evolucionar la prueba mandando parámetros en el body del POST y validamos que la respuesta es la del nuevo producto guardado.

Ya que guardar el producto involucra hacer la conexión con MongoDB, vamos a aplicar un fake, es decir, de momento vamos a retornar un objeto falso como si fuera el guardado realmente.

¿Por qué?

La idea es hacer el MÍNIMO esfuerzo para que las pruebas pasen y conforme agregue más pruebas, nos irán guiando en el diseño de la solución con lo que realmente necesitamos.

Entonces la prueba queda ahora como:

JavaScript

Copiar código 

```
describe('POST /products', () => {
  test('should store a new product', async () => {
    const response = await request(app)
      .post('/products')
      .send({
        name: 'my product',
        description: 'this is a test',
        price: 100,
        stock: 50
      })
      .set('Accept', 'application/json')
      .expect('Content-Type', /json/)
      .expect(201);

    expect(response.body).toEqual({
      name: 'my product',
      description: 'this is a test',
      price: 100,
      stock: 50,
      _id: 'abc'
    });
  });
});
```

Algunas pruebas

La prueba falla naturalmente:

```
> products-tdd@1.0.0 test
> jest

FAIL:  __tests__/products.test.js
  POST /products
    ✕ should store a new product (31 ms)

  ● POST /products › should store a new product

    expect(received).toEqual(expected) // deep equality

    Expected: {"_id": "abc", "description": "this is a test", "name": "my product", "price": 100, "stock": 50}
    Received: {"_id": "abc", "description": "this is a test", "name": "my product", "price": 100, "stock": 50}

    17 |         .expect(201);
    18 |
    19 |         expect(response.body).toEqual({
    20 |           name: 'my product',
    21 |           description: 'this is a test',
    22 |           price: 100,
    23 |           stock: 50,
    24 |         });
    at Object.toEqual ( __tests__/products.test.js:19:27)
```

Para que pase, entonces retornamos un response de acuerdo al body de la request y un `_id` extra, lo que incluye que la API soporte recibir parámetros de tipo JSON en el body.

En `server.js`:

JavaScript Copiar código

```
app.use(express.json());

app.post('/products', (req, res) => {
  const { name, description, price, stock } = req.body;

  const _id = 'abc';

  res.status(201).json({
    name,
    description,
    price,
    stock,
    _id
  });
});
```

Con estos cambios la prueba vuelve a pasar:

```
> products-tdd@1.0.0 test
> jest

PASS:  __tests__/products.test.js
  POST /products
    ✓ should store a new product (60 ms)

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   0 total
Time:        0.707 s, estimated 1 s
Ran all test suites.
```

Ya hemos logrado pasar de RED a GREEN con una primera implementación donde usamos un Fake. Ahora viene el refactor de código.

Vemos que el archivo `server.js` tiene mucha lógica: la inicialización de express, definición y lógica de manejo del endpoint, iniciar el server. Primero vamos a separar la inicialización del server y toda la demás lógica de express.

Creamos un nuevo archivo `app.js` y dentro colocamos:

JavaScript

Copiar código 

```
const express = require('express');
const app = express();

app.use(express.json());

app.post('/products', (req, res) => {
  const { name, description, price, stock } = req.body;

  const _id = 'abc';

  res.status(201).json({
    name,
    description,
    price,
    stock,
    _id
  });
});

module.exports.app = app;
```

Ahora el archivo `server.js` queda como:

JavaScript

Copiar código 

```
const { app } = require('./app');

const port = 8080;

if (require.main === module) {
  app.listen(() => console.log(`listening on port ${port}`));
}

module.exports.app = app;
```

Guardamos y validamos que las pruebas sigan pasando.

El siguiente refactor es separar la definición de endpoints y la inicialización de express. Entonces creamos un nuevo archivo `routes.js` y dentro colocamos:

JavaScript

Copiar código 

```
const express = require('express');

const router = express.Router();

router.post('/products', (req, res) => {
  const { name, description, price, stock } = req.body;

  const _id = 'abc';

  res.status(201).json({
    name,
    description,
    price,
    stock,
    _id
  });
});

module.exports.router = router;
```

Creamos una instancia de `express.Router`, la cuál nos ayuda a definir endpoints.

Importamos `router` en `app.js` y le indicamos a `express` que usaremos los endpoints que tenga definidos:

JavaScript

Copiar código 

```
const express = require('express');

const { router } = require('./routes');

const app = express();

app.use(express.json());

app.use(router);

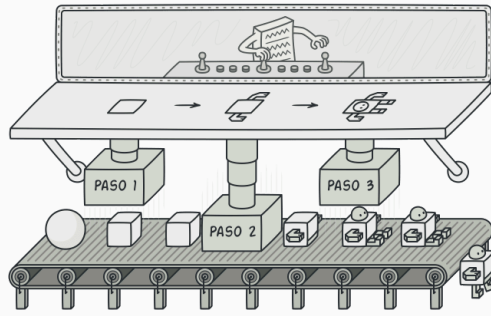
module.exports.app = app;
```

Verificamos que las pruebas sigan pasando.

Un último refactor de momento va a ser en la misma prueba. Podemos observar que estamos repitiendo los datos del producto cuando mandamos el request y cuando hacemos el expect. Lo malo de esto es que si el producto evoluciona a tener diferentes propiedades, van a haber dos lugares distintos donde tengamos que actualizar la prueba. Para que la prueba sea más fácil de mantener, vamos a aplicar el patrón builder:

El patrón builder es un patrón de diseño de tipo creacional que resuelve el problema de cómo crear instancias de objetos.

El patrón nos permite producir distintos tipos y representaciones de un objeto empleando el mismo código de construcción.



Originalmente, este patrón es usado con POO (Programación Orientada a Objetos), pero nosotros haremos una versión funcional:

JavaScript

Copiar código 

```
const Builder = {
  product: ({
    name = 'my product',
    description = 'this is a test',
    price = 100,
    stock = 50
  } = {}) => ({
    name,
    description,
    price,
    stock
  })
};
```

No es más que un objeto con la propiedad product que es una función que nos retorna a su vez un objeto con las propiedades esperadas de un producto. Recibe por parámetro un objeto con propiedades por default, de modo que si queremos especificar un nombre en particular, lo podemos hacer.

La prueba queda como:

JavaScript

Copiar código 

```
const request = require('supertest');

const { app } = require('../server');

const Builder = {
  product: ({
    name = 'my product',
    description = 'this is a test',
    price = 100,
    stock = 50
  } = {}) => ({
    name,
    description,
    price,
    stock
  })
};

describe('POST /products', () => {
  test('should store a new product', async () => {
    const product = Builder.product();
    const response = await request(app)
      .post('/products')
      .send(product)
      .set('Accept', 'application/json')
      .expect('Content-Type', /json/)
      .expect(201);

    expect(response.body).toEqual({
      ...product,
      _id: 'abc'
    });
  });
});
```

Ahora vamos a hacer un refactor para separar el Builder en un archivo independiente de manera que pueda exportar el builder en donde quiera.

En la carpeta raíz creamos una carpeta builders y dentro un archivo **product-builder.js**:

JavaScript

Copiar código 

```
// builders/product-builder.js
module.exports.Builder = {
  product: ({
    name = 'my product',
    description = 'this is a test',
    price = '100'
  } = {}) => ({
    name,
    description,
    price
  })
};
```

Y en la prueba hacemos el require:

JavaScript

Copiar código 

```
const { Builder } = require('../builders/product-builder.js');
```

Con esto hemos concluido la primera iteración del ciclo RED - GREEN - REFACTOR de TDD.

Unit tests VS Integration tests

Hemos heredado la idea de que los desarrolladores debemos enfocarnos más en hacer unit tests porque son las pruebas más rápidas y que menos esfuerzo implican. Pero conforme ha ido evolucionando el ecosistema de testing (principalmente en Javascript), hoy en día podemos crear pruebas de integración e incluso E2E con un tiempo y costo parecidos a que si hiciéramos pruebas unitarias.

Si el propósito de tener pruebas automatizadas es tener seguridad de que nuestro código funciona como esperamos que funcione con el menor esfuerzo posible, probablemente nos convenga hacer más pruebas de integración.

La clave es crear pruebas que no estén acopladas a los detalles de implementación.

Un detalle de implementación es que la prueba conozca un aspecto de cómo funciona internamente aquello que está probando. En nuestro caso, nuestro unit test recién creado está acoplado al detalle de que se debe ejecutar una función store. La desventaja de esto es que la prueba será obsoleta en el momento en que haga un refactor o cambie la implementación.

Pero si hacemos una prueba de integración donde probemos la funcionalidad sin importar los detalles de implementación, entonces tenemos mayor libertad de hacer refactors o cambios sin que las pruebas se vean afectadas. Al contrario, las pruebas de integración nos dirán si los cambios que hacemos rompen o no con lo que esperamos que haga la API. Al fin y al cabo, al usuario final no le importa si usamos Mongo u otra base de datos, le importa que pueda usar nuestra API.

Por otra parte, al tener la prueba de integración, estaremos cubriendo al mismo tiempo el funcionamiento de operaciones a base de datos (y todo lo que implique) siendo algo redundante tener pruebas unitarias sobre esa parte.

Hay una excepción aquí: si estuviéramos desarrollando software que no vaya a tener modificaciones impactantes, entonces podría valer la pena hacer muchos unit tests. Pero lo natural es que el software siempre evolucione.

Unit Testing aplicando TDD en Node

Hasta ahora nuestra implementación devuelve una respuesta hardcodeada, es decir, es una respuesta que ha sido programada de antemano en el código y no puede ser cambiada o ajustada de manera dinámica. Es una respuesta fija que no se basa en ningún tipo de entrada o cálculo, sino que se encuentra preestablecida en el código fuente.

Nos interesa guardar el producto realmente en una base de datos de Mongo.

¿Cómo podemos hacerlo con TDD?

La mejor manera de iniciar es hacer un mock de la lógica para manejar las operaciones a la base de datos. Es decir, tener las operaciones CRUD necesarias encapsuladas en funciones que podamos exportar de un archivo, de modo que en las pruebas podamos hacerles un mock y manejarlas como necesitemos.

En la prueba vamos a asumir que va a existir un archivo `service/product-service.js` que tenga de inicio una función llamada `store` (para hacer el Create del CRUD) y con Jest la usaremos como mock.

Para crear un mock de una implementación en Jest se usa `jest.mock(url/to/my/implementation)`, que en nuestro caso queda como: `jest.mock('../services/product-service.js')`.

Entonces pasamos a crear el archivo `services/product-service.js` y exportamos la función:

JavaScript

Copiar código 

```
// services/product-service.js
module.exports.store = async () => {};
```

Estamos asumiendo que la función `store` va a guardar nuestro producto en base de datos. Por lo tanto, agregamos una prueba nueva donde vamos a validar que esta función se esté ejecutando.

JavaScript

Copiar código 

```
const request = require('supertest');

const { app } = require('../server');
const { Builder } = require('../builders/product-builder.js');
const { store } = require('../services/product-service');

jest.mock('../services/product-service.js');

beforeEach(() => {
  store.mockReset();
});

describe('POST /products', () => {
  test('should store a new product', async () => {
    // code...
  });

  test('should execute store function', async () => {
    await request(app)
      .post('/products')
      .send(Builder.product())
      .set('Accept', 'application/json')
      .expect('Content-Type', /json/)
      .expect(201);

    expect(store).toHaveBeenCalled();
  });
});
```

Nota: en beforeEach estamos reseteando el estado del mock store para garantizar que en cada test es un mock con un estado limpio.

Si probamos la prueba, veremos que falla. Para hacer que pase, vamos a importar y ejecutar la función store en [routes.js](#):

JavaScript

Copiar código 

```
const express = require('express');

const { store } = require('./services/product-service');

const router = express.Router();

router.post('/products', async (req, res) => {
  const { name, description, price, stock } = req.body;

  const _id = 'abc';

  await store();

  res.status(201).json({
    name,
    description,
    price,
    stock,
    _id
  });
});

module.exports.router = router;
```

La prueba ya pasa exitosamente.

```
> products-tdd@1.0.0 test
> jest

PASS  __tests__/products.test.js
  POST /products
    ✓ should store a new product (1 ms)
    ✓ should execute store function (46 ms)

Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Snapshots:   0 total
Time:        0.581 s, estimated 1 s
Ran all test suites.
```

Pero el hecho de ejecutar la función store no nos dice si realmente estamos guardando un producto. Por ello, vamos a modificar mi prueba para validar que esté siendo ejecutada con un parámetro igual a un objeto con las propiedades del producto. En el test ya tenemos la constante product, por lo que lo podemos utilizar como sigue.

```
test('should execute store function', async () => {
  const product = Builder.product();

  await request(app)
    .post('/products')
    .send(product)
    .set('Accept', 'application/json')
    .expect('Content-Type', /json/)
    .expect(201);

  expect(store).toHaveBeenCalledWith(product);
});
```

Y modificamos la implementación de routes:

JavaScript

Copiar código 

```
router.post('/products', async (req, res) => {
  const { name, description, price, stock } = req.body;

  const _id = 'abc';

  await store({ name, description, price, stock });

  res.status(201).json({
    name,
    description,
    price,
    stock,
    _id
  });
});
```

Ahora si, la prueba pasa exitosamente.

Lo que acabamos de hacer puede ser considerado un unit test porque estamos probando nuestro endpoint como si fuese una función aislada de services.

Vemos que la función store aún no hace nada, pero en la prueba actual estamos asumiendo que el producto se guarda en la base de datos por el hecho de que se está ejecutando dicha función. Para solucionar lo anterior tenemos al menos dos opciones:

- Crear una prueba unitaria nueva para services/product-service.js y validar aisladamente las operaciones de guardado haciendo mock de mongoose.
- Crear una prueba de integración para el endpoint POST /products donde valide el guardado en base de datos como parte de la prueba del endpoint en su totalidad.

Para decidir qué es mejor, vamos a ver los pros y contras de cada una.

Integration Testing aplicando TDD en Node

Ahora necesitamos crear una prueba de integración donde ejecutemos un endpoint y guarde un producto sin usar mocks. Esta vez queremos validar que realmente se está guardando un producto en una base de datos real.

Para este caso, tenemos la opción de hacer la implementación obvia (real) de iniciar una conexión a base de datos, o un fake y triangulación para poco a poco ir dando con la solución. Esta parte es interesante porque la idea es poder crear una función para conectarse a una base de datos que sirva para las pruebas y para la implementación real.

Aquí es donde entra `mongodb-memory-server`;

Esta herramienta nos va a permitir crear una base de datos real de mongo en memoria durante las pruebas, y al terminirlas, cerraremos la conexión y destruiremos la base de datos. MongoDB Memory Server es una herramienta que permite crear una instancia de MongoDB en la memoria RAM. Esto significa que los datos no se almacenan en disco, sino en la memoria del sistema.

La lógica va a ser:

- Si estamos ejecutando una prueba, creamos la base de datos temporal y nos conectamos a ella por medio de la uri.
- Si no es una prueba, nos conectamos a la base de datos que se indique por medio de su uri también.

En ambos casos necesitamos la uri, por lo que es buena idea pasar ese valor por parámetro.

Para comenzar, necesitamos indicar a la prueba que debe hacer una conexión a una base de datos antes de ejecutar las pruebas.

De antemano, aún no existe esa lógica, pero sabemos que tendremos una función `connect` y `getUri`.

JavaScript

Copiar código 

```
const { connect, getUri } = require('../db');
```

Y agregamos un `beforeAll` para ejecutar la conexión.

JavaScript

Copiar código 

```
beforeAll(async () => {  
  const uri = await getUri();  
  await connect({ uri });  
});
```

Naturalmente, nos da un error porque ni siquiera hemos creado el archivo. Creamos el archivo `db/index.js` y comenzamos por importar las dependencias:

JavaScript

Copiar código 

```
const mongoose = require('mongoose');
const { MongoMemoryServer } = require('mongodb-memory-server');

mongoose.Promise = Promise;

const mongoServer = new MongoMemoryServer();
```

Establecemos que Mongoose utilizará las promesas de JavaScript en lugar de las callbacks, y creamos una nueva instancia de MongoMemoryServer. Esta instancia se utilizará para crear una conexión a una instancia de MongoDB en la memoria, que se utilizará para las pruebas.

Luego exportamos la función `getUri`:

JavaScript

Copiar código 

```
module.exports.getUri = async () => {
  if (process.env.NODE_ENV === 'test') {
    await mongoServer.start();
    return mongoServer.getUri();
  }
  return process.env.DB_URI;
};
```

Aquí está la parte clave. Si `process.env.NODE_ENV === 'test'` es true, significa que estamos ejecutando una prueba, ya que Jest actualiza `process.env.NODE_ENV` igual a test automáticamente.

Dentro obtenemos la uri necesaria para crear una conexión por medio de la función `mongoServer.getUri()`.

Si no es una prueba, entonces se salta el if y retorna lo que tenga la variable de entorno `process.env.DB_URI`, que más adelante haremos que tenga un valor por medio de un archivo `.env`.

Ahora exportamos la función `connect`:

JavaScript

Copiar código 

```

module.exports.connect = async ({ uri }) => {
  const mongooseOpts = {
    useUnifiedTopology: true,
    useNewUrlParser: true
  };

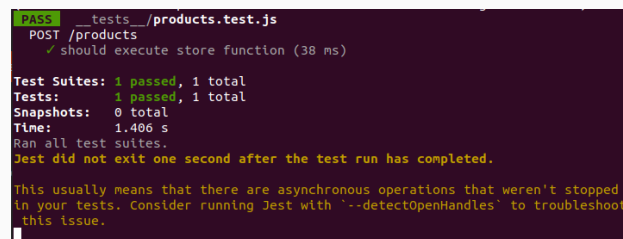
  await mongoose.connect(uri, mongooseOpts);

  mongoose.connection.once('open', () => {
    console.log(`MongoDB successfully connected to ${uri}`);
  });
};

```

A esta función no le importa si estamos en un entorno de pruebas o no, ya que sólo va a recibir la uri a la cuál conectarse.

Perfecto, con estos cambios las pruebas pasan, pero ahora tenemos un warning:



```

PASS  tests/products.test.js
  POST /products
    ✓ should execute store function (38 ms)

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:  0 total
Time:        1.406 s
Ran all test suites.
Jest did not exit one second after the test run has completed.
This usually means that there are asynchronous operations that weren't stopped
in your tests. Consider running Jest with '--detectOpenHandles' to troubleshoot
this issue.

```

Esto es debido a que necesitamos cerrar y destruir la base de datos temporal.

En la prueba importamos una nueva función `closeDb` y la ejecutamos en un `afterAll`:

JavaScript

Copiar código 

```

const { connect, getUri, closeDb } = require('../db');
//...
afterAll(async () => {
  await closeDb();
});

```

Naturalmente, la prueba falla de nuevo porque aún no existe esa función. Procedemos a crearla en `db/index.js`:

JavaScript

Copiar código 

```

module.exports.closeDb = async () => {
  await mongoose.disconnect();

  if (process.env.NODE_ENV === 'test') {
    await mongoServer.stop();
  }
};

```


Vamos a modificar la prueba dejando de hacer mock a la función store porque ahora pretendemos guardar realmente el producto en una base de datos. Por lo tanto, el segundo test queda obsoleto por lo que lo borramos también.

La prueba final queda como:

JavaScript

Copiar código 

```
const request = require('supertest');

const { app } = require('../server');
const { connect, getUri, closeDb } = require('../db');
const { Builder } = require('../builders/product-builder.js');

beforeAll(async () => {
  const uri = await getUri();
  await connect({ uri });
});

afterAll(async () => {
  await closeDb();
});

describe('POST /products', () => {
  test('should store a new product', async () => {
    const product = Builder.product();

    const response = await request(app)
      .post('/products')
      .send(product)
      .set('Accept', 'application/json')
      .expect('Content-Type', /json/)
      .expect(201);

    expect(response.body).toEqual({
      ...product,
      _id: 'abc'
    });
  });
});
```

Verificamos que siga pasando el test.

Ya que usaremos la implementación real de guardado en base de datos, el `_id` será generado por mongo cuando se inserte un documento nuevo, por lo que cambiamos la prueba como sigue:

JavaScript

Copiar código 

```
test('should store a new product', async () => {
  const product = Builder.product();

  const response = await request(app)
    .post('/products')
    .send(product)
    .set('Accept', 'application/json')
    .expect('Content-Type', /json/)
    .expect(201);

  const { _id, ...productStored } = response.body;

  expect(productStored).toEqual(product);
  expect(_id).toBeTruthy();
});
```

Lo que hacemos es deestructurar el `response.body` para extraer el `_id` y el resto de las propiedades (el `...productStored`). Comparamos que `productStored` sea igual a `product` que es lo que mandamos por POST al endpoint.

Vamos a hacer la implementación real de la función `store`:

JavaScript

Copiar código 

```
const mongoose = require('mongoose');
const productSchema = mongoose.Schema({
  name: String,
  description: String,
  price: Number,
  stock: Number
});

const Product = mongoose.model('products', productSchema);

module.exports.store = async ({ name, description, price }) => {
  const product = new Product({
    name,
    description,
    price
  });
  await product.save();
  return product;
};
```

En este archivo definimos el modelo de mongoose y en la función `store` hacemos una instancia del modelo y el guardado en Mongo.

Verificamos que mis pruebas sigan pasando exitosamente. Ahora la prueba de integración está funcionando correctamente para un endpoint que hace un guardado en base de datos.

Con esto hemos logrado pasar de una prueba unitaria, a una prueba de integración exitosamente.

Estructura de Node JS con Express aplicando MVC

Vamos a experimentar la ventaja de tener pruebas automatizadas haciendo un refactor grande en la api: cambiar su estructura a MVC.

En el caso de esta API, tendremos modelos y controladores únicamente debido a que no manejamos vistas.

Se recomienda en todo momento tener las pruebas corriendo y hacer cambios pequeños para validar que sigan pasando, de forma tal que, si algo deja de pasar, sabremos que fue por el último cambio realizado.

Primero haremos la parte de modelo en `models/product-model.js`:

JavaScript

Copiar código 

```
const mongoose = require('mongoose');

const productSchema = mongoose.Schema({
  name: String,
  description: String,
  price: Number,
  stock: Number
});

module.exports = mongoose.model('products', productSchema);
```

Y el service en `services/product-service.js` ahora queda como:

JavaScript

Copiar código 

```
const mongoose = require('mongoose');

const productSchema = mongoose.Schema({
  name: String,
  description: String,
  price: Number,
  stock: Number
});

module.exports = mongoose.model('products', productSchema);
```

Vemos que las pruebas siguen pasando.

Ahora vamos a desacoplar las rutas y sus handlers creando los controladores en `controllers/product-controller.js`.

JavaScript

Copiar código 

```
const { store } = require('../services/product-service');

module.exports.createProduct = async (req, res) => {
  const { name, description, price, stock } = req.body;

  const _id = 'abc';

  await store({ name, description, price, stock });

  res.status(201).json({
    name,
    description,
    price,
    stock,
    _id
  });
};
```

Y el archivo de rutas queda como:

JavaScript

Copiar código 

```
const express = require('express');

const { createProduct } = require('./controllers/product-controller');

const router = express.Router();

router.post('/products', createProduct);

module.exports.router = router;
```

Comprobamos que los tests siguen pasando correctamente. Y con esto ya tenemos la API funcionando correctamente, con sus pruebas automatizadas y una estructura que permite escalarla mejor.